信息学竞赛培训 · C++ 程序设计

LESSON 04 / STAGE ONE

Sorting Algorithms in Depth

排序算法深入

04

sort 自定义比较器 · 结构体排序 · 稳定性 · vector 入门

> 排序题真正的难点从来不在 sort 本身，而在于你能否说清楚：谁该排在谁前面。把题面里的规则读出来，写成一行比较器，答案自然浮现。

04

授课班级
中预 2026 级 B 班

主讲
信息教研组

时间
2026 年 10 月

10 道经典例题 + 5 道 vector 练习

学习目标与课堂安排

第 4 课 · 排序算法深入

本节课你将学会

1 **会用 sort 的三种调用方式**

默认升序、加 greater 降序、自己写比较器——同一条语句换个参数，规则就从「从小到大」变成「按你的意思」。

2 **会用结构体打包数据、按多关键字排序**

把学号、成绩、时间这些「同属一条记录」的字段装进一个结构体，再把它整体交给 sort。

3 **认识 vector：怎么用、好在哪**

动态数组不需要事先猜大小，随读随存，还能直接配合 sort 使用——竞赛里最常用的容器。

4 **能从题面读出「关键字优先级」**

总分相同看语文、语文相同看学号……把这类自然语言，逐条翻译成比较器里的判断。

课堂安排

建议节奏，可按学情调整

20 分钟	三块基础讲解
15 分钟	插入排序与 P7910
70 分钟	十道例题精讲
40 分钟	vector 练习五题
15 分钟	总结答疑

本课主打一句话

看懂题目要「按什么排」，再动手写比较器。

顺序想清楚了，代码往往只有几行。

本课共 63 页，其中例题与练习占 45 页，末页附十道题目的自主练习清单。

承上启下

第 3 课我们学会「把所有可能试一遍」；这一课把顺序本身变成工具——先排好，再让答案自己浮出来。

本课路线图：三块基础 + 十五道题

第 4 课 · 排序算法深入

从语法到解题的完整路径

第一站 · 三块基础

sort / 结构体 / vector

把三件工具的使用方法、常见坑一次讲透，每块都配一页「重点需要关注的内容」。

9 页

第二站 · 十道经典例题

由易到难，每题三页

题目演示 → 思路拆解 → 参考代码。从排序去重一直走到动态维护名次。

30 页

第三站 · vector 练习

3 道入门 + 2 道结合

先用三道小题熟悉 vector，再把 vector 与排序组合起来解决两个实际问题。

15 页

十道经典例题（由易到难）

01 P1059 明明的随机数 —— 排序去重（★）

02 P2676 超级书架 —— 排序 + 贪心（★）

03 P1093 奖学金 —— 多关键字排序（★★）

04 P1104 生日 —— 四级关键字 + 稳定性（★★）

05 P1068 分数线划定 —— 名次并列（★★）

06 P1781 宇宙总统 —— 大数比较（★★）

07 P5143 攀爬者 —— 结构体 + 几何距离（★★★）

08 P1204 挤牛奶 —— 区间合并（★★★）

09 P1583 魔法照片 —— 两轮排序 + 取模（★★★）

10 P7910 插入排序 —— 动态维护名次（★★★★）

五道 vector 练习

V1 变长数组求和与最值 —— vector 遍历（★）

V2 第 k 小的数 —— vector + sort（★）

V3 分数段统计 —— vector + 条件计数（★）

V4 合并两个有序序列 —— vector + 归并（★★）

V5 按绝对值排序 —— vector + 自定义规则（★★）

为什么单练 vector?

竞赛里的数据规模常常「题目才告诉你」，用定长数组要么开小越界、要么开大浪费。vector 随读随存、自带长度，配合 sort 就是最顺手的组合。前五题先把它用熟，后两题再与排序合练。

sort 是什么：一行搞定排序


```cpp
#include <iostream>
#include <algorithm>      // sort 住在这个头文件里
using namespace std;

int a[1005];

int main() {
  int n;
  cin >> n;
  for (int i = 0; i < n; i++) cin >> a[i];

  sort(a, a + n);          // 把 a[0] 到 a[n-1] 从小到大排好

  for (int i = 0; i < n; i++) cout << a[i] << " ";
  return 0;
}
```

一句话理解 sort

sort 就是「把一段区间里的元素，按你给的规则重新排列」。不给规则，就默认从小到大。

两个参数 vs 三个参数

sort(起点, 终点) 用默认规则；sort(起点, 终点, 比较规则) 用你给的规则。起点与终点是「位置」（指针或迭代器），不是长度。

区间是左闭右开

sort(a, a+n) 排的是下标 0 到 n-1 这 n 个元素，终点的 a+n 本身不在区间内。这是 C++ 里通用的区间习惯。

性能放心用

sort 平均复杂度 O(n log n)，10 万个数不到 0.01 秒。竞赛里只要需要「有序」，第一反应就该是 sort。

sort 的三种调用方式


```
int a[1005];

// 写法 ① 默认升序：从小到大
sort(a, a + n);

// 写法 ② 降序：加一个 greater 比较器
sort(a, a + n, greater<int>());

// 写法 ③ 自定义规则：自己写比较函数
bool cmp(int x, int y) {
  return x > y;         // 返回 true，表示 x 应排在 y 前面
}
sort(a, a + n, cmp);    // 效果与写法 ② 相同
```

① 默认升序

不写第三个参数，sort 就用「小于」做比较，结果是从小到大。绝大多数场合够用。

② greater 降序

greater 是标准库现成的「大于」比较器，加上它就从大到小。注意它后面那对空括号不能省。

③ 自定义比较器

cmp(x, y) 返回 true，表示 x 应该排在 y 前面。想按什么规则排，就在这个函数里写清楚。

比较器到底在回答什么?

它回答的是：「第一个参数该不该排在第二个参数前面？」返回 true 就让它上前，false 就保持。

★ sort 重点需要关注的内容

把下面五条过一遍，sort 的坑基本就绕开了。

01 **传位置，不传长度**

sort(a, a+n) 的两个参数是「起始位置」和「结束位置」，不是数组名加长度。写成 sort(a, n) 编译就过不去。

02 **区间左闭右开**

排的是从 a 到 a+n 这一段，共 n 个元素，终点那一格不参与排序。

03 **比较器必须「严格弱序」**

比较函数里只能用小于或大于，绝不能写小于等于、大于等于。用错会破坏排序的前提，轻则结果错乱，重则程序崩溃。

04 **sort 不保证稳定**

值相等的两个元素，排完后谁前谁后是不确定的。要让相等元素保持原来的先后次序，请改用 stable_sort，写法完全一样。

05 **vector 也能直接排**

对 vector 排序要传迭代器：sort(v.begin(), v.end())。区间同样是左闭右开。

结构体是什么：把一条记录打包


```cpp
#include <iostream>
using namespace std;

// 定义一个「学生」类型：把同属一条记录的字段打包
struct Stu {
  int id;      // 学号
  int chn;     // 语文
  int total;   // 总分
};

Stu s[305];    // 305 个学生，每个都自带三个字段

int main() {
  s[1].id = 1;
  s[1].chn = 90;
  s[1].total = 265;

  cout << s[1].total << "
";  // 用点号访问成员
  return 0;
}
```

什么是结构体

把若干个「属于同一条记录」的变量，打包成一个新类型。就像给一张表格的一行起个名字。

怎么定义、怎么用

用 struct 关键字定义，大括号里列出各字段。使用时用变量名加点号访问成员，例如 s[1].total。

为什么排序题离不开它

排序会交换元素。若字段分散在几个数组里，交换时要同时搬家、极易出错；打包成一个结构体，一次交换就搞定。

命名建议

结构体名用大写开头（Stu、Node），变量名用小写，一眼就能区分「类型」和「变量」。

结构体排序：多关键字怎么写


```cpp
struct Stu {
  int id;
  int total;
  int chn;
};
Stu s[305];

// 总分降序 → 语文降序 → 学号升序
bool cmp(const Stu &a, const Stu &b) {
  if (a.total != b.total) return a.total > b.total;
  if (a.chn   != b.chn)   return a.chn   > b.chn;
  return a.id < b.id;
}

int main() {
  int n; cin >> n;
  for (int i = 1; i <= n; i++) {
    int c, m, e; cin >> c >> m >> e;
    s[i].id = i;
    s[i].chn = c;
    s[i].total = c + m + e;
  }
  sort(s + 1, s + n + 1, cmp);   // 整体排序，交换的是整个结构体
  return 0;
}
```

比较器参数用 const 引用

写成 const Stu &a，既避免复制整个结构体，也防止误改原数据。这是结构体比较器的标准写法。

多关键字 = 依次判断

从第一优先级开始：不相等就立刻定胜负；相等才继续看下一级。最后一级仍相等，两人的先后对答案无影响。

排序交换的是整个结构体

你不需要手动搬运各字段。sort 一动，id、成绩、时间这些字段会跟着一起走。

升降序别写反

降序用大于号，升序用小于号。写完一定拿样例验一遍——方向写反是这类题最常见的失分点。

★ 结构体重点需要关注的内容

结构体本身不难，难在「用对」。下面五条是历年题里翻车最多的地方。

01 **比较器一定要传 const 引用**

写成 const Stu &a 的形式。若按值传参，每次比较都会复制整个结构体，数据量大时白白拖慢程序。

02 **关键字的先后顺序不能颠倒**

总分 → 语文 → 学号，顺序就是你判断的顺序。写反了，样例往往还能过，隐藏数据就挂了。

03 **方向容易写反，务必验样例**

降序用大于、升序用小于。多关键字里经常升降混用，写完逐条对着题面再核一遍。

04 **结构体可以整体赋值与传参**

s[i] = s[j] 会一次性复制全部字段。把结构体传给函数时，同样建议用引用，避免无谓复制。

05 **排序写法与普通数组完全一致**

只要提供比较器，sort(s + 1, s + n + 1, cmp) 就能用。下标从 1 开始时，别把那个 +1 漏了。

vector 是什么：会自己变长的数组


```cpp
#include <iostream>
#include <vector>          // vector 在这个头文件里
using namespace std;

int main() {
  vector<int> v;          // 一个空的 int 动态数组

  v.push_back(3);         // 在末尾追加元素
  v.push_back(1);
  v.push_back(4);         // 现在 v = [3, 1, 4]

  cout << v.size() << "
";   // 3：元素个数
  cout << v[0] << "
";       // 3：下标从 0 开始

  for (int i = 0; i < (int)v.size(); i++)
    cout << v[i] << " ";      // 3 1 4
  return 0;
}
```

vector 是什么

一个「长度可以变」的数组。不用一开始就决定开多大，读到多少就存多少，读完了也不会浪费空间。

最常用的四个操作

push_back(x) 末尾追加；size() 取元素个数；v[i] 按下标访问；v.clear() 清空。记住这四个就够起步。

下标同样从 0 开始

和普通数组一样，第一个元素是 v[0]，最后一个元素是 v[v.size() - 1]。

别忘了头文件

vector 与 algorithm 这两个头文件都要写在最上面：前者提供 vector，后者提供 sort。缺一个都会编译报错。

vector 的优势：竞赛里为什么人人都在用

同样是「存一串数」，vector 比定长数组多解决了三个麻烦：不知道有多少个、中途还要加、不想管内存。

相比定长数组，vector 好在哪

① 长度可变 —— 读到多少就存多少，不必事先猜一个「够大」的数字。

② 不易越界 —— 定长数组开小了，是新手程序崩溃最常见的原因。

③ 自带长度 —— v.size() 随时告诉你现在有几个元素，不用另记变量。

④ 自动管理内存 —— 空间不够时自动扩容，你只管往末尾追加。

⑤ 与算法无缝配合 —— sort(v.begin(), v.end()) 一行排好整串。

什么时候「必须」用 vector

题目没有直接告诉你数据个数，而是「读到文件结束为止」；

或者要在运行过程中不断往容器里加数据——这两种情况，定长数组都不好用。

补充：v.size() 返回的是无符号整数，与 0 比较没有问题；

但做减法时要留神，下一节「重点需要关注的内容」会专门讲。

读入任意多个数，再排序

```cpp
#include <iostream>
#include <vector>
#include <algorithm>
using namespace std;

int main() {
  vector<int> v;
  int x;

  // 一直读到没有输入为止，有多少存多少
  while (cin >> x) v.push_back(x);

  sort(v.begin(), v.end());    // 整串排序

  for (int i = 0; i < (int)v.size(); i++)
    cout << v[i] << " ";
  return 0;
}
```

读到文件末尾自动停止。竞赛里凡是没有明确给出「一共几个数」的题，这种写法最省心。

注意排序时传的是 v.begin() 与 v.end() 两个迭代器，不是数组名。

对照记忆：数组用 sort(a, a+n)，vector 用 sort(v.begin(), v.end())，两者都是「左闭右开」。

★ vector 重点需要关注的内容

vector 用起来简单，但下面五条几乎每一条都有人踩过。

01

size() 是无符号整数

当 vector 为空时，v.size() 减 1 会变成一个很大的正整数，循环条件千万要小心，否则直接越界。

02

比较时加 (int) 转换

循环条件写成 i 小于 (int)v.size() 更稳妥，也能消掉编译器的有符号与无符号比较警告。

03

追加元素用 push_back

往 vector 里加东西就用 v.push_back(x)。不要写 v[5] = x 去「凭空赋值」——那是越界，行为未知。

04

下标区间仍是 0 到 size-1

第一个元素 v[0]，最后一个 v[v.size() - 1]。越界访问不会被检查，后果由自己承担。

05

与 sort 配合要传迭代器

sort(v.begin(), v.end()) 就是给整个 vector 排序。不要写成 sort(v, v + v.size())。

插入排序：为什么它天然稳定


```cpp
// 插入排序：把 a[0] 到 a[n-1] 从小到大排好
for (int i = 1; i < n; i++) {
  int key = a[i];              // 摸到的新牌
  int j = i - 1;
  while (j >= 0 && a[j] > key) {  // 比 key 大的，统统往后挪一格
    a[j + 1] = a[j];
    j--;
  }
  a[j + 1] = key;              // 空出来的位置就是 key 的家
}
```

一个生活类比

打扑克摸牌：左手已是一把排好的牌，摸到新牌就往里插。插入排序做的正是这件事。

为什么它天然稳定

内层循环的条件是「严格大于才往后挪」。遇到相等的元素就停下，新牌插在等值元素后面，相对次序得以保持。

复杂度与适用场景

平均和最坏都是 $O(n^2)$，n 到几千还行，上万就慢了。它的价值在于「稳定」与「接近有序时很快」。

这条循环条件是关键

把内层的「大于」误写成「大于等于」，排序结果看着一样，但稳定性被破坏——P7910 正是靠这个性质解题的。

T01 | 明明的随机数（P1059）

第 4 课 · 排序算法深入

NOIP2006 普及组 · 难度 ★

明明用随机数生成了 n 个整数，其中有些是重复的。请把重复的去掉，再把剩下的从小到大排序，最后输出个数与排好序的数列。

输入格式

第一行一个正整数 n，表示生成的随机数个数；第二行 n 个用空格隔开的正整数，即生成的随机数。

输出格式

共两行：第一行一个整数，表示去重后剩下几个数；第二行按从小到大的顺序输出这些数，相邻两个数之间用一个空格隔开。

数据范围

n ≤ 100；每个随机数不超过 1000。这两个上限都极小，说明本题不考性能，只考「去重 + 排序」两件事怎么干净地做出来。

样例

```
10
20 40 32 67 40 20 89 300 400 15
```

```
8
15 20 32 40 67 89 300 400
```

输入 10 个数，其中 20 与 40 各重复了一次，去重后剩 8 个。
排序后按 15、20、32、40、67、89、300、400 依次输出。
先排序再去重、或先去重再排序，都能得到同样的结果。

本题关键

① 这是「排序 + 去重」最经典的入门模型：数据里有重复，重复的只保留一份。
② sort 之后相等的元素必然相邻——只要和它的前一个比较，相同就跳过。这是去重的通用套路。
③ 别忘了先输出剩下的个数，再输出数列；个数写错是本题最常见的失分点。

思路拆解：先排序，再去重

两个子任务有先后依赖：排序让相同的数挤在一起，去重才能一次扫描搞定。

读入并排序

把 n 个数读进数组，直接 sort 升序。这一步是上一节学过的标准调用，本身没有难度。

为什么排序能帮我们去重

排序之后所有相等的数一定挨在一起。于是「判断这个数是不是重复的」就退化成「判断它和左边相邻那个数是否相等」——一次比较就够。

边扫描边保留

从第一个元素开始遍历，只做一件事：当前元素与它前一个不同，就保留。第一个元素必然保留，可以单独处理，也可以在数组前放一个不会出现的哨兵值。

注意输出格式

题目要求第一行输出个数、第二行输出数列。建议先把去重结果存进新数组，最后统一输出，数目与数列不容易对不上。

一句话记住

「排序 + 相邻比较」是处理重复元素最省事的组合。哈希去重虽然更快，但出来的顺序是乱的；本题要求升序输出，排序法反而更直接。

参考代码：排序 + 相邻判重


```cpp
#include <iostream>
#include <algorithm>
using namespace std;

int a[105];

int main() {
  int n;
  cin >> n;
  for (int i = 0; i < n; i++) cin >> a[i];

  sort(a, a + n);            // 排序后相同的数一定相邻

  int m = 0;
  for (int i = 0; i < n; i++)
    if (i == 0 || a[i] != a[i - 1])   // 只留第一次出现的
      a[m++] = a[i];

  cout << m << "
";
  for (int i = 0; i < m; i++) {
    if (i) cout << " ";
    cout << a[i];
  }
  cout << "
";
  return 0;
}
```

相邻判重只比较一次

升序序列里相同元素必然连续，所以只要和 a[i-1] 比。这就是「利用有序性」的典型思路，比一个个回头搜快得多。

原地复用同一个数组

去重结果直接写回 a 的前 m 个位置，省掉一个额外数组。因为 m 永远不会超过 i，不会覆盖还没读到的数据。

空格的处理技巧

if (i) 先输出一个空格，可以让第一个元素前面不带空格。很多评测系统对此并不严格，但养成习惯总是好的。

提交提示

官方样例已通过验证。若本机输出看着对却评测不过，优先检查空格与换行的位置是否和题目要求一致。

T02 | 超级书架（P2676）

第 4 课 · 排序算法深入

USACO · 难度 ★

书架前面有一排奶牛，第 i 头奶牛身高为 hi，踩在它们背上就能叠成一个阶梯。要够到高度 B 的书，最少需要几头奶牛？

输入格式

第一行两个整数 n 与 B，分别表示奶牛头数和书架高度；接下来 n 行，每行一个整数 hi，表示第 i 头奶牛的身高。

输出格式

一行一个整数，表示最少需要几头奶牛叠起来才能达到或超过高度 B。

数据范围

n ≤ 20000；B ≤ 2×10⁹；hi ≤ 10⁹。数据规模大、身高也大，所以必须用「排序后贪心」，不能暴力枚举子集。

样例

```
6 40
6
18
11
13
19
11
```

```
3
```

身高为 6、18、11、13、19、11，书架高 40。

从高到低取：19 + 18 = 37 还不够，再加 13 得到 50，够了，共 3 头。

换成更矮的组合只会用更多头，所以每次都挑最高的最划算。

本题关键

① 目标是用最少的头数堆够高度，直觉上应当从最高的开始选——这就是贪心，且本题贪心是正确的。

② 做法很直接：把身高降序排序，从头累加，一旦达到或超过 B 就停，输出已用的头数。

③ B 可达 2×10⁹，累加变量必须用 long long，用 int 会溢出。这是本题最容易丢分的地方。

思路拆解：为什么「从高到低」是最优的

用交换论证说服自己：贪心策略为什么不会漏掉更优解。

先想清楚目标

题目不关心是哪几头奶牛，只关心用了多少头。所以让每头都尽可能高，就能在更少的头数里凑够高度。

交换论证

假设存在一个最优方案，其中一头身高为 h，而我们贪心对应选的那头身高为 H 且 H 不小于 h。把它换成更高的那头，总高度只会增加，够得到书架，头数没有变多。反复替换，贪心方案也是最优方案。

推论：直接排序贪心

既然每一步都该选当前最高的，那把全部身高降序排好、从头累加就是最优顺序，不必反复挑最大值。

细节：累加与溢出

累加到和达到 B 就立刻停止并输出头数。累加变量声明为 long long，防止 2×10^9 级别的数据把 int 撑爆。

一句话记住

「用最少的个数凑够总量」这类题，第一反应就是把候选值从大到小排序再贪心。排序是贪心最常用的前置步骤。

参考代码：降序排序 + 贪心累加


```cpp
#include <iostream>
#include <algorithm>
using namespace std;
long long h[20005];

// 从大到小排序
bool cmp(long long a, long long b) { return a > b; }

int main() {
  int n;
  long long B;              // 书架高度要用 long long
  cin >> n >> B;
  for (int i = 0; i < n; i++) cin >> h[i];
  sort(h, h + n, cmp);      // 降序：大的排在前面

  long long sum = 0;
  for (int i = 0; i < n; i++) {
    sum += h[i];
    if (sum >= B) {         // 够了就立刻输出头数
      cout << i + 1 << "
";
      return 0;
    }
  }
  cout << n << "
";      // 兜底：全部用上
  return 0;
}
```

降序比较器的写法

cmp 返回 a 大于 b，sort 就会把大的排在前面，与默认升序正好相反。这是本题排序部分唯一需要注意的地方。

long long 的两处使用

B 与 sum 都必须用 long long。身高最大 10^9，n 最大 20000，全加起来远超 int 上限，一旦溢出结果就是随机的。

够到就立刻返回

在循环里直接 return，避免引入额外的标志变量。这种「找到答案就早退」的写法在贪心题里非常常见，也更简洁。

提交提示

官方样例已通过验证。请特别确认 B 与 sum 的类型是 long long —— 这道题就是用来提醒你注意数据范围与溢出风险的。

T03 | 奖学金（P1093）

第 4 课 · 排序算法深入

NOIP2007 普及组 · 难度 ★★

按总分从高到低排序；总分相同按语文从高到低；总分与语文都相同，则学号小的排在前面。输出前 5 名的学号与总分。

输入格式

第一行 n（学生人数）；接下来 n 行，每行三个整数表示语文、数学、英语成绩。学号按输入顺序为 1 到 n。

输出格式

共 5 行，每行两个整数：学号 总分，按题目规则排序后的前 5 名。

数据范围

n ≤ 300；每科成绩 0 到 100。输出必须严格按规则排序——顺序错了不给分。

样例

```
6
90 67 80
87 66 91
78 89 91
88 99 77
67 89 64
78 89 98
```

```
6 265
4 264
3 258
2 244
1 237
```

第 6 号总分 78+89+98 = 265 最高；第 4 号 88+99+77 = 264 第二。

三个关键字本题全部用得上，是「多关键字排序」最标准的练习题。

注意学号是升序，与总分、语文的降序方向相反。

本题关键

① 总分要现算：读入时顺手求出三科之和，省得排序时反复重复计算。

② 三个关键字的优先级：总分 → 语文 → 学号，顺序不能颠倒。

③ 学号是升序（小的在前），与总分、语文的降序方向相反——这是最容易写反的一处。

思路拆解：三条规则的翻译

把题面里的三条规则逐条翻译成比较器里的判断，一条对应一行代码。

第一步 · 把学号带上

排序后还要输出学号，而学号不会随结构体自动记住，所以必须把它作为一个字段存进去。用结构体把学号与成绩打包在一起。

第二步 · 先算总分

读入时就把三科相加存进 total 字段。不要在比较器里现算——比较器会被调用成千上万次，重复计算纯属浪费。

第三步 · 按优先级写判断

第一级：总分不等，总分高的在前（降序）。第二级：总分相等但语文不等，语文高的在前。第三级：两者都相等，学号小的在前（升序）。

第四步 · 取前五名输出

排序完成后，输出前 5 个元素的学号与总分即可。n 不超过 300，直接排序完全够快。

一句话记住

多关键字的通用写法是：从第一优先级开始逐个判断——不相等就直接给出结果，相等才继续看下一级。最后一级判断完仍相等，说明两个元素在所有可见维度上完全一样，它们之间的次序对答案无影响。

参考代码：结构体 + 多关键字比较器


```cpp
#include <iostream>
#include <algorithm>
using namespace std;

struct Stu { int id, chn, total; };   // 学号、语文、总分
Stu s[305];

// 总分降序 → 语文降序 → 学号升序
bool cmp(const Stu &a, const Stu &b) {
  if (a.total != b.total) return a.total > b.total;
  if (a.chn   != b.chn)   return a.chn   > b.chn;
  return a.id < b.id;
}

int main() {
  int n; cin >> n;
  for (int i = 1; i <= n; i++) {
    int c, m, e; cin >> c >> m >> e;
    s[i].id = i; s[i].chn = c;
    s[i].total = c + m + e;     // 读入时就把总分算好
  }
  sort(s + 1, s + n + 1, cmp);
  for (int i = 1; i <= 5; i++)
    cout << s[i].id << " " << s[i].total << "
";
  return 0;
}
```

结构体只存需要的字段

数学与英语在排序中从未被单独使用，因此不必存进结构体——只有语文是第二关键字，需要保留。字段越少，出错机会越少。

比较器三级判断

三条规则三条 if，顺序与优先级一一对应。注意最后一级是学号升序，方向与前两级相反。

下标从 1 开始

学号与下标天然对应，用 1 基下标可以省掉「输出 i+1」这类容易忘的操作。数组开到 305 留出余量。

提交提示

本题两组官方样例均已通过验证。注意输出的是「学号 总分」，不是名次——请对照样例检查输出格式。

T04 | 生日（P1104）

第 4 课 · 排序算法深入

普及− · 难度 ★★

按年龄从大到小排序并输出姓名。特别注意：如果有两个同学年龄（出生年月日）完全相同，输入靠后的同学先输出。

输入格式

第一行 n（人数）；接下来 n 行，每行是姓名 年 月 日。姓名不含空格，数据保证日期真实存在。

输出格式

共 n 行，每行一个姓名，按年龄从大到小的顺序输出。

数据范围

n ≤ 100；年份在 1900 到 2000 之间。数据很小，本题考的是「稳定性」这个概念，而不是算法效率。

样例

```
3
Yangchu 1992 4 23
Qiujingya 1993 10 13
Luowen 1991 8 1
```

```
Luowen
Yangchu
Qiujingya
```

Luowen 生于 1991 年，最早出生，年龄最大，排第 1。
Yangchu 1992 年次之，Qiujingya 1993 年最小。
三人日期互不相同，因此本样例只用到前三级关键字。

本题关键

① 年龄大等于出生日期早。所以按「年升序、月升序、日升序」排序，越早出生越靠前。
② 关键陷阱：出生日期相同时，输入靠后的先输出——这是一条「反稳定」的规则，不能依赖排序算法本身的行为。
③ 解决办法：把输入顺序存进结构体的一个字段，作为最后一级关键字，并让它的方向与输入顺序相反。

思路拆解：为什么必须自己处理「输入顺序」

这道题的价值在于：它逼着你正面理解「稳定性」这个概念。

观察一 · 排序方向比较绕

年龄大的人出生早，所以比较时「年小的排前面」。三个日期字段的优先级是年 → 月 → 日，逐级判断，写法与上一题完全同构。

观察二 · 相同日期的规则是反的

题目要求「输入靠后的先输出」——这恰好与稳定排序的行为相反（稳定会保持输入顺序，即靠前的先输出）。所以直接依赖排序算法是不行的。

观察三 · 把顺序变成字段

给结构体加一个 order 字段记录输入序号。比较的最后一级写成：order 大的排前面。这样规则就被显式写进了比较器，不依赖任何算法的默认行为。

观察四 · 一级一级往下判断

年不等比年 → 月不等比月 → 日不等比日 → 都比完了就比 order（大的在前）。四级判断，逻辑一目了然。

一句话记住

排序算法是否稳定，取决于它的实现；而题目要求的顺序规则，永远应该由你自己在比较器里写明。把需要的信息（如输入序号）主动放进结构体，是让排序「完全听话」的最可靠做法。

参考代码：四级关键字比较器


```cpp
#include <iostream>
#include <string>
#include <algorithm>
using namespace std;
struct Person {
  string name;
  int y, m, d;
  int order;          // 输入顺序（第几个读入的）
};
Person p[105];

// 年龄大在前（年/月/日 升序）；日期相同则输入靠后的在前
bool cmp(const Person &a, const Person &b) {
  if (a.y != b.y) return a.y < b.y;
  if (a.m != b.m) return a.m < b.m;
  if (a.d != b.d) return a.d < b.d;
  return a.order > b.order;    // 关键：顺序大的在前
}

int main() {
  int n; cin >> n;
  for (int i = 1; i <= n; i++) {
    cin >> p[i].name >> p[i].y >> p[i].m >> p[i].d;
    p[i].order = i;           // 记下输入顺序
  }
  sort(p + 1, p + n + 1, cmp);
  for (int i = 1; i <= n; i++) cout << p[i].name << "
";
  return 0;
}
```

order 字段的作用

它把「输入顺序」从一种隐含信息变成显式数据。这样无论用什么排序算法，规则都由你说了算。这种「把信息搬进结构体」的思路在很多题里都能用。

为什么最后一级是大于号

前三级都是小的在前（升序），最后一级要「输入靠后的在前」，也就是序号大的在前，所以用大于号——方向相反正是本题的考点。

string 的读入

cin 读 string 遇到空格或换行即停止，正好符合「姓名不含空格」的输入格式，比字符数组安全得多。

提交提示

官方样例已通过验证。请务必自己造一组「出生日期完全相同」的数据自测——那才是这道题真正想考的地方。

T05 | 分数线划定（P1068）

第 4 课 · 排序算法深入

NOIP2009 普及组 · 难度 ★★

按笔试成绩从高到低排序，成绩相同则报名号小的在前。面试分数线取第 m×150%（向下取整）名选手的成绩；所有成绩不低于该线的选手都能进入面试。

输入格式

第一行两个整数 n 与 m，表示报名人数与计划录取人数；接下来 n 行，每行两个整数：报名号 k 与笔试成绩 s。

输出格式

第一行两个整数：面试分数线 与 进入面试的实际人数；随后逐行输出进入面试者的报名号与成绩，按成绩降序、成绩相同则报名号升序。

数据范围

n ≤ 5000；m ≤ n；报名号 1000 到 9999 且互不相同；成绩 1 到 100。

样例

```
6 3
1000 90
3239 88
2390 95
7231 84
1005 95
1001 88
```

```
88 5
1005 95
2390 95
1000 90
1001 88
3239 88
```

3×150% = 4.5，向下取整为 4，所以分数线是第 4 名的成绩 88。

由于 88 有并列，成绩不低于 88 的共 5 人，因此人数是 5 而不是 4。

输出时两位 95 分按报名号升序：1005 在前、2390 在后。

本题关键

① 两个关键字：成绩降序、成绩相同则报名号升序。比较器写法和前面几题完全同构。

② 分数线看的是「第 floor(m×1.5) 名」的成绩，而不是简单取前 floor(m×1.5) 个人。

③ 并列会让人数变多：分数线上所有同分的人都必须进入面试。

思路拆解：分数线与并列的处理

排序本身很常规，真正容易错的是「分数线取谁的分数」与「并列怎么办」。

第一步 · 排序

按成绩降序、成绩相同报名号升序，把 n 个人排好。这一步是本课最熟悉的套路。

第二步 · 定位分数线

算出 k 等于 m 乘 1.5 并向下取整。排好序后第 k 个人的成绩就是面试分数线。注意 k 是从 1 开始数的名次。

第三步 · 处理并列

从第 k 名往后看，只要成绩仍等于分数线，就要一并算进面试名单。因此实际人数可能大于 k，必须一直延伸到成绩低于分数线为止。

第四步 · 按序输出

先输出分数线与实际人数，再按排好的顺序逐行输出进入面试者的报名号与成绩。

易错点

① 分数线取的是「第 k 名的成绩」，不是「前 k 个人」。

② 并列会让人数增加，输出的人数要与实际名单长度一致，不能直接写 k。

③ 输出必须严格按成绩降序、报名号升序，顺序错了不给分。

参考代码：结构体 + 并列延伸


```cpp
#include <iostream>
#include <algorithm>
using namespace std;

struct Person { int id, score; };
Person p[5005];

// 成绩降序；成绩相同则报名号升序
bool cmp(const Person &a, const Person &b) {
  if (a.score != b.score) return a.score > b.score;
  return a.id < b.id;
}

int main() {
  int n, m; cin >> n >> m;
  for (int i = 0; i < n; i++) cin >> p[i].id >> p[i].score;
  sort(p, p + n, cmp);

  int k = m * 3 / 2;            // m × 1.5 向下取整
  int line = p[k - 1].score;    // 第 k 名的成绩即分数线

  while (k < n && p[k].score == line) k++;  // 并列者一并进入

  cout << line << " " << k << "
";
  for (int i = 0; i < k; i++)
    cout << p[i].id << " " << p[i].score << "
";
  return 0;
}
```

整数除法实现向下取整

m 是正整数，m 乘 3 再整除 2 恰好等于 m 乘 1.5 向下取整，不需要浮点数，也避免了精度问题。

分数线是取「成绩」

p[k-1].score 是第 k 名的分数，把它当作线；所有成绩不低于它的人都进入面试。这是本题最容易理解偏的一步。

并列用 while 延伸

从 k 开始往后扫，成绩仍等于分数线就一直 k 加一。注意循环条件要同时判断越界与分数相等，顺序不能反。

提交提示

官方样例已通过验证。请重点核对两处：m×1.5 的取整、以及并列延伸后输出的人数是否与实际名单一致。

T06｜宇宙总统（P1781）

第 4 课 · 排序算法深入

普及− · 难度 ★★

有 n 个人参加选举，每人的票数是一个可能非常长的整数。票数最多的人当选，请输出他的编号与票数。

输入格式

第一行一个整数 n，表示候选人数；接下来 n 行，每行一个大整数，表示该候选人的得票数。

输出格式

共两行：第一行是当选者的编号（从 1 开始）；第二行是该候选人的票数，按输入原样输出。

数据范围

n ≤ 20；票数不超过 100 位。关键点在于票数远远超出 long long 的范围，必须当作字符串处理。

样例

```
5
98765
12365
87954
1022356
985678
```

```
4
1022356
```

五人票数分别是 98765、12365、87954、1022356、985678。

1022356 是七位数，其余都是五到六位，位数最多者最大。

因此编号 4 当选，输出 4 与 1022356。

本题关键

① 票数可达 100 位，远超 64 位整数的表示范围，用 long long 一定溢出，必须按字符串处理。

② 字符串比较是字典序，对长度不同的数字串并不适用：「985678」按字典序比「1022356」大，但数值上更小。

③ 正确做法：先比位数，位数多的数更大；位数相同时再比字典序，此时字典序恰好等价于数值大小。

思路拆解：为什么不能直接比字符串

这道题不考排序，考的是「超大数比较」这一基础能力。

问题 · 整数存不下

100 位的十进制数需要约 333 个二进制位，而 long long 只有 63 位有效位。所以只能用字符串来存。

陷阱 · 字典序不等于数值序

字符串比较是逐字符按编码比。数字字符的编码递增，所以在长度相同时字典序确实等价于数值序；但长度不同时就会出错。

解法 · 先比长度，再比字典序

一个十进制数若没有前导零，位数多的必然更大。所以第一步比较长度；只有长度相同时才用字典序比较。两级规则合起来就等价于数值比较。

实现 · 边读边更新最大值

不需要把所有票数都存下来。读入一个就比较一次，记录当前最大的串与其编号，最后输出即可，空间是常数级。

一句话记住

处理超出整数范围的数，标准套路就是「当作字符串 + 先比长度再比字典序」。这个技巧在竞赛里反复出现，值得直接记住并复用。

参考代码：字符串比大小


```cpp
#include <iostream>
#include <string>
using namespace std;

int main() {
  int n; cin >> n;
  string best = "";       // 当前最大票数
  int bestId = 0;

  for (int i = 1; i <= n; i++) {
    string s; cin >> s;
    // 去前导零，避免影响位数判断
    while (s.size() > 1 && s[0] == '0') s.erase(0, 1);

    if (bestId == 0) { best = s; bestId = i; continue; }

    bool bigger = false;
    if (s.size() > best.size()) bigger = true;       // 位数多者更大
    else if (s.size() == best.size() && s > best)    // 位数相同比字典序
      bigger = true;

    if (bigger) { best = s; bestId = i; }
  }

  cout << bestId << "
" << best << "
";
  return 0;
}
```

为什么要去前导零

若输入写成 007，字符串长度是 3；而另一个 10 长度是 2。不去前导零会误判 007 更大。去掉后长度为 1，比较才正确。

两级判断缺一不可

先比长度，长度相同再比字典序。只比字典序会出错，只比长度也无法区分同位数的大小，两者必须配合使用。

不需要保存全部数据

边读边更新最大值，空间是常数级。这种做法在处理大数据量时尤其有用，也是「在线算法」的最简单形式。

提交提示

官方样例已通过验证。可以自己造一组数据验证陷阱，例如三个数 9、10、100，正确输出应当是编号 3 与 100。

T07 | 攀爬者（P5143）

第 4 课 · 排序算法深入

普及− · 难度 ★★★

在三维空间里有 n 个点，攀爬者要从最低的点依次爬到最高的点。求这条必须按高度递增经过的路线总长度。

输入格式

第一行一个整数 n，表示点的个数；接下来 n 行，每行三个整数 x、y、z，表示一个点的坐标。

输出格式

一行一个实数，表示攀爬路线的总长度，保留三位小数。

数据范围

$n \le 50000$；坐标绝对值不超过 10^4。注意 z 是高度，路线必须严格按 z 从小到大经过所有点。

样例

```
5
2 2 2
1 1 1
4 4 4
3 3 3
5 5 5
```

```
6.928
```

五个点按高度 z 排序后依次是 (1,1,1)、(2,2,2)、(3,3,3)、(4,4,4)、(5,5,5)。

相邻两点距离都是根号 3，约 1.7320508，四段合计约 6.9282。

输入顺序是打乱的，这正是本题要考的地方。

本题关键

① 输入给出的点顺序是乱的，必须先按高度 z 升序排序，才能得到合法的攀爬次序。

② 排序的对象是「点」，所以要用结构体把三个坐标打包在一起，让它们作为一个整体被交换。

③ 距离公式是三维欧氏距离：各坐标差的平方和再开方。记得包含 cmath 或 math 头文件。

④ 输出保留三位小数，用 printf 的百分号点三 f 格式，别用 cout 的默认精度。

思路拆解：为什么必须把坐标打包成结构体

这道题的核心不是距离计算，而是「多个字段作为一个整体排序」这一需求。

痛点 · 三个数组会被拆散

如果分别用 x、y、z 三个数组存坐标，直接对 z 排序只会打乱高度，x 与 y 还留在原地，点与点之间的对应关系就被破坏了。

解法 · 用结构体绑定

把 x、y、z 放进同一个结构体，数组的每个元素就是一个完整的点。排序时交换的是整个点，三个坐标始终绑在一起，不会错位。

比较器 · 只按高度比

cmp 只需要比较两个点的 z：z 小的在前。其他字段不参与比较，但仍然随着结构体一起移动，这正是我们想要的。

计算 · 排序后累加相邻距离

排好序后，从第 1 个点到第 n 个点依次累加相邻两点的三维距离，最后按要求输出三位小数即可。

一句话记住

只要「排序的键」和「要输出的数据」不是同一个字段，就应该考虑用结构体把它们捆在一起。这是结构体排序存在的根本理由，也是它在竞赛题里出现频率最高的原因。

参考代码：结构体 + 按高度排序


```
#include <iostream>
#include <algorithm>
#include <cmath>
#include <cstdio>
using namespace std;
struct Point { int x, y, z; };   // z 为高度

bool cmp(const Point &a, const Point &b) {
  return a.z < b.z;              // 按高度升序
}

Point p[50005];
int main() {
  int n; cin >> n;
  for (int i = 0; i < n; i++)
    cin >> p[i].x >> p[i].y >> p[i].z;

  sort(p, p + n, cmp);           // 排完序即为攀爬次序

  double total = 0.0;
  for (int i = 1; i < n; i++) {
    double dx = p[i].x - p[i-1].x;
    double dy = p[i].y - p[i-1].y;
    double dz = p[i].z - p[i-1].z;
    total += sqrt(dx*dx + dy*dy + dz*dz);
  }
  printf("%.3f
", total);       // 保留三位小数
  return 0;
}
```

结构体把坐标绑在一起

数组 p 的每个元素是一个完整的点。排序交换的是整个结构体，x、y、z 永远同步移动，这正是本题必须用结构体的原因。

比较器只看 z

cmp 只比较 z 字段。排序后的下标顺序就是攀爬次序，之后按这个顺序累加即可，无需再记录原编号。

距离要用 double 存

dx、dy、dz 声明为 double，避免整数相减后再平方造成的隐患。sqrt 返回 double，累加到 total 上即可。

提交提示

官方样例已通过验证。请用 printf 的百分号点三 f 输出，别用 cout 默认精度，否则会因小数位数不符而判错。

T08 | 挤牛奶（P1204）

第 4 课 · 排序算法深入

USACO · 难度 ★★★

有 n 个农民分别在各自的时段挤牛奶，时段之间可能重叠。求「至少有一人挤奶」的最长连续时间，以及「无人挤奶」的最长连续空档。

输入格式

第一行一个整数 n，表示农民人数；接下来 n 行，每行两个整数 l 与 r，表示该农民从 l 时刻挤到 r 时刻。

输出格式

一行两个整数，用空格隔开：第一是最长的连续挤奶时间，第二是最长的连续无人挤奶时间。

数据范围

$n \le 5000$；时刻范围 0 到 10^6。时段可能互相包含、部分重叠或完全分离，三种情况都要正确处理。

样例

```
3
300 1000
700 1200
1500 2100
```

```
900 300
```

前两段 [300,1000] 与 [700,1200] 重叠，合并后覆盖 [300,1200]，长度 900。
第三段从 1500 开始，与前面的覆盖区之间空出 1500 减 1200 等于 300。
因此答案是两个数：900 与 300。

本题关键

① 这是「区间合并」的经典模型。核心观察是：把所有时段按左端点升序排序，重叠的区间必然会被排到一起，一次扫描即可合并完。
② 扫描时维护一个「当前覆盖区间」，若下一个区间的左端点不超过当前右端点，说明二者相接或重叠，把右端点扩展到两者的较大值。
③ 若下一个区间的左端点大于当前右端点，说明出现断档：先结算，再开启新段。

思路拆解：一次扫描同时求两个答案

区间合并的模板：排序之后，相邻关系就足以判断重叠与断裂。

第一步 · 按左端点排序

把 n 个区间按左端点升序排好。排序后，左端点小的区间一定排在前面，重叠关系只可能在相邻区间之间发生。

第二步 · 维护当前覆盖段

用两个变量记录当前正在合并的覆盖区间的左右端点，初始时取第一个区间的左右端点。

第三步 · 逐段判断两种情形

对后面的每个区间：若它的左端点不大于当前右端点，说明相连或重叠，把右端点更新为两者的较大值；否则说明断裂，先结算当前覆盖长度、更新最长挤奶时间，再用新左端点减去当前右端点得到空隙、更新最长空档，最后把当前段换成新区间。

第四步 · 收尾结算

循环结束后，最后一个覆盖段还没有参与比较，必须再结算一次它的长度。漏掉这一步会导致答案偏小。

一句话记住

区间合并的标准套路是「按左端点排序 + 维护当前右端点」。只要题目涉及「时段重叠」「覆盖长度」「空档」，几乎都是这个模板，值得完整背下来。

参考代码：排序 + 区间合并


```cpp
#include <iostream>
#include <algorithm>
using namespace std;
struct Seg { int l, r; };
bool cmp(const Seg &a, const Seg &b) {
  return a.l < b.l;              // 按左端点升序
}

Seg s[5005];
int main() {
  int n; cin >> n;
  for (int i = 0; i < n; i++) cin >> s[i].l >> s[i].r;
  sort(s, s + n, cmp);

  int cl = s[0].l, cr = s[0].r;   // 当前覆盖区间
  int bestCov = 0, bestGap = 0;
  for (int i = 1; i < n; i++) {
    if (s[i].l <= cr) {
      if (s[i].r > cr) cr = s[i].r;     // 合并：右端点取较大值
    } else {
      // 断档：先结算两个极值，再开启新段
      if (cr - cl > bestCov) bestCov = cr - cl;
      if (s[i].l - cr > bestGap) bestGap = s[i].l - cr;
      cl = s[i].l; cr = s[i].r;
    }
  }
  if (cr - cl > bestCov) bestCov = cr - cl;  // 收尾，别漏

  cout << bestCov << " " << bestGap << "
";
  return 0;
}
```

合并时取较大值

只有新区间右端点更大时才更新 cr。这一句处理了「新区间被旧区间包含」的情况；若直接赋值，覆盖段会缩短，是本题最隐蔽的坑。

断档时结算两次

出现断档意味着一整段覆盖结束、一段空隙产生，所以要同时更新两个极值。注意顺序：先用旧右端点算空隙，再改左右端点。

收尾结算不可省

循环结束后，最后一整段覆盖还没有与任何后续区间比较过。补一句结算，答案才完整。

提交提示

官方样例已通过验证。建议自己再构造「一个区间完全包含另一个」的数据自测，确认合并时用了取最大值的写法。

T09｜魔法照片（P1583）

第 4 课 · 排序算法深入

普及− · 难度 ★★★

n 个人各有权值。先按权值从大到小排一次定出「序号」，再按序号对 10 取模加上额外权值 E，最后按新权值重新排序，输出前 k 个人的编号。

输入格式

第一行两个整数 n 与 k；第二行 10 个整数，即 E_1 到 E_{10}；第三行 n 个整数，表示每个人的初始权值 W。

输出格式

一行，按最终权值从大到小的顺序输出前 k 个人的编号（编号即输入顺序 1 到 n），相邻用空格隔开。

数据范围

$n \le 20000$；$k \le n$；所有权值不超过 10^9。权值很大且要经历两次排序，比较规则必须写准确。

样例

```
10 10
1 2 3 4 5 6 7 8 9 10
2 4 6 8 10 12 14 16 18 20
```

```
10 9 8 7 6 5 4 3 2 1
```

初始权值升序为 2、4、…、20，第一次排序后序号第 1 名是权值 20 的人（编号 10）。

加 E 后权值分别为 20+1、18+2、16+3……恰好仍保持降序，因此最终输出编号从大到小。

本题两轮排序的关键字与优先级都不一样，是本课最综合的一道。

本题关键

① 题目的「两轮排序」顺序绝对不能颠倒：先用初始权值排一次，目的是确定每个人的序号。

② 序号从 1 开始计数。第 pos 名的人加上的是 E 数组里的第 ((pos − 1) 模 10) + 1 个元素，下标换算极易写错。

③ 两轮排序在权值相同时都要求编号小的排在前面，比较器规则一致。

思路拆解：两轮排序与下标换算

本题的难点不在排序本身，而在「序号」这个中间量的传递与取模换算。

第一轮 · 定出每个人的序号

按初始权值降序、权值相同则编号升序，排序一次。排在第 pos 位的人，序号就是 pos，从 1 开始数。

关键 · 序号要能传到第二轮

加权紧跟第一轮排序之后、数组顺序尚未被打乱，此时直接用循环变量 pos 就是序号。若中间还要插入别的操作，就必须把序号固化进结构体。

加权的下标换算

序号 pos 对应的是 E 数组的第 ((pos − 1) 对 10 取余) 再加 1 个元素。因为 pos 从 1 起、数组下标从 1 起，两处偏移都要照顾到。

第二轮 · 按新权值重排

用更新后的权值再排一次序，规则与第一轮相同（权值降序、编号升序），最后输出前 k 个元素的编号。

一句话记住

只要某次排序的中间结果需要在后续步骤里用到，就应该把它固化进结构体的一个字段。结构体字段越多，逻辑越清晰——这比用多个平行数组去维护对应关系可靠得多。

参考代码：两轮排序 + 取模加权


```cpp
#include <iostream>
#include <algorithm>
using namespace std;
typedef long long ll;
struct Node { ll w; int id; };
Node a[20005];
int E[11];

// 权值降序；权值相同则编号升序（两轮规则一致）
bool cmp(const Node &x, const Node &y) {
  if (x.w != y.w) return x.w > y.w;
  return x.id < y.id;
}

int main() {
  int n, k; cin >> n >> k;
  for (int i = 1; i <= 10; i++) cin >> E[i];
  for (int i = 1; i <= n; i++) { cin >> a[i].w; a[i].id = i; }
  sort(a + 1, a + n + 1, cmp);        // 第一轮：定出序号
  for (int pos = 1; pos <= n; pos++)
    a[pos].w += E[(pos - 1) % 10 + 1];  // 序号取模后加权
  sort(a + 1, a + n + 1, cmp);        // 第二轮：按新权值重排

  for (int i = 1; i <= k; i++) {
    if (i > 1) cout << " ";
    cout << a[i].id;
  }
  cout << "
";
  return 0;
}
```

两轮比较器内容相同

本题两条排序规则一致，所以复用同一个 cmp。若题目中两轮规则不同，就分别写两个比较函数、在两次 sort 里传不同的即可。

取模的下标换算

pos 从 1 起、E 从下标 1 起，所以写成 (pos 减 1) 对 10 取余再加 1。这样 pos 等于 1 时取到 E_1，pos 等于 11 时又回到 E_1。

权值要用 long long

初始权值可达 10^9，再加上 E 之后仍在 10^9 量级，但两轮比较涉及相加，用 long long 更稳妥，避免边界处溢出。

提交提示

官方样例已通过验证。请重点核对取模换算与「权值相同看编号」这两处，它们是本题被判错的主要原因。

T10 | 插入排序（P7910）

第 4 课 · 排序算法深入

CSP-J 2021 第二轮 · 难度 ★★★★

H 老师给了一个长度为 n 的数组，需要支持 q 次操作：把某个元素改成新值，或询问某个元素按稳定插入排序后会排在第几位。

输入格式

第一行 n 与 q；第二行 n 个整数表示初始数组；接下来 q 行，每行为「2 x」表示询问，或「1 x v」表示把第 x 个元素改成 v。

输出格式

对每一次询问操作输出一行一个整数，表示该元素在从小到大稳定排序后的名次（从 1 开始）。

数据范围

n ≤ 8000，q ≤ 8000，修改次数不超过 5000。若每次询问都重新排序，复杂度会爆，必须用「维护名次 + 局部调整」。

样例

```
3 4
3 2 1
2 3
1 3 2
2 2
2 3
```

```
1
1
2
```

初始数组 3 2 1。询问第 3 个元素排完在第几位，答案是 1（它是 1，最小）。

接着把第 3 个元素改成 2，数组变成 3 2 2，之后两次询问分别得到 1 和 2。

注意：值相同的两个元素仍看作不同元素——去路不同，答案就不同。

本题关键

① 稳定排序的含义：值相同时，原来下标小的排在前面。题目的插入排序天然满足这一点。

② 关键洞察：修改一个元素的值，其余元素的相对次序完全没有变，只有这一个元素需要重新找位置。

③ 于是把「整体重排」降级为「让一个元素向左或向右冒泡归位」，每次修改的代价从 n log n 降到最坏 O(n)。

思路拆解：只挪一个元素

这是本课最难的一道，但核心想法只有一句话：改了一个数，就只把这个数挪回它该在的地方。

第一步 · 建立初始名次

把 n 个原始下标按「值升序、值相同下标升序」排一次。用一个数组记住排序后第 k 位是谁，再用一个数组记住每个元素当前排第几。这一步是 O(n log n)，只做一次。

第二步 · 询问变成一次读表

询问某个元素的位置时，直接查它当前的名次数组即可，O(1)。这就是优化带来的最大收益。

第三步 · 修改只挪一个元素

把某个元素改值后，它的位置可能变了，但不会偏太远：向左挪（比左邻该靠前就交换）或向右挪（比右邻该靠后就交换）几下，就能重新归位。

第四步 · 同步更新名次表

每次交换两个位置时，必须把两个元素的名次也一起改过来。两个数组互为反函数，漏改一个，下次询问就会答错。

复杂度核算

初始化 O(n log n)，只需一次。每次修改最坏挪 n 步，修改不超过 5000 次，总计约 4×10^7 次操作，完全可过。而朴素做法每次询问都要重排，量级高得多。

参考代码：维护名次 + 局部调整


```cpp
#include <iostream>
#include <algorithm>
using namespace std;
int a[8005], idx[8005], rk[8005];
int n, q;
// 值小的在前；值相同则原下标小的在前（与稳定排序一致）
bool cmp(int i, int j) {
  if (a[i] != a[j]) return a[i] < a[j];
  return i < j;
}
int main() {
  cin >> n >> q;
  for (int i = 1; i <= n; i++) cin >> a[i];
  for (int i = 1; i <= n; i++) idx[i] = i;
  sort(idx + 1, idx + n + 1, cmp);     // 一次排序，定出初始名次
  for (int k = 1; k <= n; k++) rk[idx[k]] = k;
  while (q--) {
    int op; cin >> op;
    if (op == 2) { int x; cin >> x; cout << rk[x] << "
"; continue; }
    int x, v; cin >> x >> v;
    a[x] = v; int k = rk[x];
    while (k > 1 && cmp(idx[k], idx[k-1])) {   // 往左挪
      swap(idx[k], idx[k-1]); rk[idx[k]] = k; rk[idx[k-1]] = k - 1; k--;
    }
    while (k < n && cmp(idx[k+1], idx[k])) {   // 往右挪
      swap(idx[k], idx[k+1]); rk[idx[k]] = k; rk[idx[k+1]] = k + 1; k++;
    }
  }
  return 0;
}
```

cmp 就是「稳定性」的代码化

先比值、值相同再比原下标——这一行保证我们的名次结果与题目给出的插入排序完全一致。少了第二级，样例就可能出错。

两个数组互为反函数

idx 记录「第 k 位是谁」，rk 记录「谁排第几」。交换 idx 的两个位置后，必须把两个元素的新名次写回 rk，否则下次询问读到过期数据。

两段 while 不会来回反复

一个元素只可能比原来更小（往左）或更大（往右），因此两个方向各扫一遍就够，不会出现来回震荡。

提交提示

官方样例已通过验证。重点自查三点：cmp 是否带上了原下标的比较、swap 后是否同步更新了 rk、数组是否开到了 8005 以上。

V01 | 变长数组求和与最值

第 4 课 · 排序算法深入

vector 入门练习 ① · 难度 ★

读入 n 个整数，把它们存进 vector，然后输出所有数的和与最大值。这是熟悉 vector 基本操作的第一道热身题。

输入格式

第一行一个整数 n；第二行 n 个用空格隔开的整数。

输出格式

一行两个整数，用空格隔开：所有数的和 与 其中的最大值。

数据范围

$n \le 100000$；每个整数的绝对值不超过 10^9。n 很大，但直接遍历一遍即可，考的是 vector 的读入与遍历。

样例

```
5
3 7 2 9 4
```

```
25 9
```

五个数依次是 3、7、2、9、4，和为 25，最大的是 9。

注意最大值不要随手初始化为 0——若有负数就会出错，应当取第一个元素作初值。

本题关键

① 定义 vector 后，用 push_back 一个个往末尾加，读到几个就存几个。

② 遍历时循环条件写成 i 小于 (int)v.size()，加 (int) 是为了避开无符号比较的警告。

③ 求最大值的初值取 v[0]，而不是 0 —— 这是新手最常见的一个隐蔽错误。

思路拆解：vector 的三步走

vector 的用法可以归成三步：定义、装入、遍历。这道题把三步都走了一遍。

第一步 · 定义

声明一个存放整数的 vector，起个名字叫 v，就得到一个空的动态数组。它现在长度是 0，等着你往里加东西。

第二步 · 装入

循环 n 次，每次读入一个数并 push_back 到末尾。也可以一开始就把长度定成 n（构造时传参，或调用 resize），再按下标读入。

第三步 · 遍历

用下标从 0 走到 v.size() 减 1，边求和边更新最大值。这是 vector 最常用的遍历方式，与普通数组完全一致。

验证 · 边界情况

把 n 取 1 试一试：此时和与最大值都是那唯一的元素。程序应当仍然正确，不需要特殊处理。

一句话记住

vector 的下标用法与普通数组一模一样，区别只在于长度是动态的、可以用 size() 随时问。凡是「数组能用」的地方，vector 基本都能直接替换，而且更安全。

参考代码：vector 读入与遍历


```cpp
#include <iostream>
#include <vector>
using namespace std;

int main() {
  int n;
  cin >> n;

  vector<int> v;
  for (int i = 0; i < n; i++) {
    int x; cin >> x;
    v.push_back(x);        // 读到几个就存几个
  }

  long long sum = 0;       // 和可能很大，用 long long
  int mx = v[0];           // 最大值取首元素作初值
  for (int i = 0; i < (int)v.size(); i++) {
    sum += v[i];
    if (v[i] > mx) mx = v[i];
  }

  cout << sum << " " << mx << "
";
  return 0;
}
```

push_back 与预分配

本题用 push_back 逐个追加，最直观。若元素个数已知，也可以构造时直接指定长度 n，再按下标逐个读入，省掉一次扩容。

和要用 long long

n 可达 10 万、每个数可达 10^9，总和能到 10^{14} 量级，int 一定溢出。看到「求和」就要下意识检查类型。

最大值初值别取 0

如果所有数都是负数，用 0 作初值会得到错误答案。取第一个元素作初值最稳妥，也是一行就能写好的事。

提交提示

本题样例已通过验证。若要自己检查，可以试一组全负数的数据（如三个 -5 -2 -9），正确输出应为 -16 与 -2。

V02 | 第 k 小的数

第 4 课 · 排序算法深入

vector 入门练习 ② · 难度 ★

给 n 个整数和一个整数 k，输出这 n 个数中第 k 小的数。这是 vector 与 sort 的第一次正式配合。

输入格式

第一行两个整数 n 与 k；第二行 n 个用空格隔开的整数。

输出格式

一行一个整数，表示第 k 小的数。

数据范围

$n \le 100000$；k 从 1 到 n；每个数的绝对值不超过 10^9。排序一次即可，允许重复数字参与计数。

样例

```
6 3
7 2 9 4 1 5
```

```
4
```

六个数排序后是 1、2、4、5、7、9。

第 3 小就是排在第三位的 4。

注意「第 k 小」对应下标 k 减 1，因为下标从 0 开始——这是本题唯一的坑。

本题关键

① 最简单的做法：整体排序一次，然后直接取下标 k 减 1 的元素。

② 对 vector 排序时，传的是 v.begin() 与 v.end() 这一对迭代器，不是 v 与 v 加长度。

③ 下标换算要留意：第 1 小是 v[0]，第 k 小是 v[k - 1]。

思路拆解：把「第 k 小」翻译成下标

这道题的思路只有一句话，难点全在下标换算与排序调用上。

第一步 · 装进 vector

按题目给出的 n 读入所有数。若个数已知，直接开好 n 个位置再读入会更省事。

第二步 · 整体排序

调用 sort(v.begin(), v.end())，让序列从小到大排好。这一步是 O(n log n)，10 万个数毫无压力。

第三步 · 下标换算

排序后第 1 小的元素在 v[0]，所以第 k 小的元素在 v[k - 1]。这类「名次转下标」的换算每次都要想一遍，别凭感觉写。

第四步 · 输出

直接输出 v[k - 1] 即可，不需要额外的循环。

一句话记住

「第 k 小」与「数组下标」之间差 1，而「第 k 小」与「第 k 大」之间不仅差 1，还要换个方向。凡遇到名次类的题，先把这两个换算在纸上写清楚，再动手。

参考代码：vector + sort


```cpp
#include <iostream>
#include <vector>
#include <algorithm>
using namespace std;

int main() {
  int n, k;
  cin >> n >> k;

  vector<int> v(n);        // 预先开好 n 个位置
  for (int i = 0; i < n; i++) cin >> v[i];

  sort(v.begin(), v.end());  // 从小到大排序

  cout << v[k - 1] << "
"; // 第 k 小对应下标 k - 1
  return 0;
}
```

两种读入方式

构造时直接传入长度 n，就一次开好 n 个位置，之后可以像数组一样按下标读入。若个数未知，才用 push_back 逐个追加。

迭代器不能写错

sort(v.begin(), v.end()) 是对的；写成 sort(v, v + v.size()) 会编译不过。记住「容器用迭代器、数组用指针」这条对照。

允许重复元素

本题的「第 k 小」是按排序后的位置数的，重复的数各占一个位置。若题意是「第 k 个不同的数」，就要先去重，那是另一回事。

提交提示

本题样例已通过验证。可以再试 k 等于 1 与 k 等于 n 两个边界，分别应当输出最小的数和最大的数。

V03 | 分数段统计

第 4 课 · 排序算法深入

vector 入门练习 ③ · 难度 ★

读入 n 个学生的分数，统计三个分数段各有多少人：不及格（低于 60）、及格（60 到 79）、优秀（80 及以上）。

输入格式

第一行一个整数 n；第二行 n 个用空格隔开的整数，表示每个人的分数，均在 0 到 100 之间。

输出格式

一行三个整数，用空格隔开：不及格人数 及格人数 优秀人数。

数据范围

n ≤ 100000；分数 0 到 100。本题不需要排序，考的是遍历与多分支判断的写法。

样例

```
8
55 72 88 100 63 45 79 91
```

```
2 3 3
```

低于 60 的有 55 与 45，共 2 人。
60 到 79 的有 72、63、79，共 3 人。
80 及以上的有 88、100、91，共 3 人，合计 8 人正好对上。

本题关键

① 三个计数器初始化为 0，遍历一遍、按分数落到哪一段就加一，不需要排序。
② 分支顺序很重要：先判断「低于 60」，再判断「低于 80」，剩下的自然就是 80 及以上。
③ 分界点是 60 与 80 —— 60 属于及格段、80 属于优秀段。写完拿边界值验一遍。

思路拆解：一次遍历 + 三个计数器

这道题的套路在竞赛里非常常见：给一批数据，按区间分类计数。

第一步 · 读入

把 n 个分数读进 vector。这里不关心顺序，所以读入之后不需要做任何排序。

第二步 · 准备计数器

开三个整型变量，分别对应不及格、及格、优秀，初值都是 0。变量名起得清楚一点，后面不容易弄混。

第三步 · 一次遍历分类

对每个分数，用 if 与 else if 链判断它落在哪一段，对应计数器加一。注意条件顺序：从下界开始卡，最后一个 else 兜住剩下的。

第四步 · 按序输出

三个计数器按题目要求的顺序输出。分界点的归属要特别核对：60 算及格、80 算优秀。

一句话记住

「分区间计数」的标准写法是「一条 if 链 + 若干计数器」。条件顺序一旦写反（比如先判断小于 80），就会把不及格的人也归进及格段，这是这类题最典型的错误。

参考代码：遍历 + 条件计数


```cpp
#include <iostream>
#include <vector>
using namespace std;

int main() {
  int n;
  cin >> n;

  vector<int> v;
  for (int i = 0; i < n; i++) {
    int x; cin >> x;
    v.push_back(x);
  }

  int low = 0, mid = 0, high = 0;
  for (int i = 0; i < (int)v.size(); i++) {
    if (v[i] < 60)      low++;    // 不及格
    else if (v[i] < 80) mid++;    // 及格
    else                high++;   // 优秀
  }

  cout << low << " " << mid << " " << high << "
";
  return 0;
}
```

if 链的顺序

先判 v[i] 小于 60，再判小于 80，最后 else 兜住。这样 60 落到第二段、80 落到第三段，与题意的分界完全一致。

为什么不用排序

本题只关心每段有多少人，与顺序无关。排序是多余的一步，白费 O(n log n) 的时间。做题前先想清楚「到底需不需要有序」。

计数器初始化

三个变量必须显式初始化为 0。局部变量不会自动清零，忘记初始化会得到随机结果，这类错误在多组测试里极难排查。

提交提示

本题样例已通过验证。建议再试一组只有边界值的数据：60、80、59、79，正确输出应当是 1 2 1。

V04 | 合并两个有序序列

第 4 课 · 排序算法深入

vector 与排序结合 ① · 难度 ★★

给定两个已经从小到大排好序的序列，把它们合并成一个仍然有序的序列并输出。这是「归并」思想的最小模型。

输入格式

第一行两个整数 n 与 m；第二行 n 个升序整数；第三行 m 个升序整数。

输出格式

一行，按从小到大的顺序输出合并后的所有数，相邻两个数之间用一个空格隔开。

数据范围

n、m ≤ 100000。两个序列都已经有序，这个条件就是用来省时间的——不利用它就可惜了。

样例

```
3 4
1 3 5
2 4 6 8
```

```
1 2 3 4 5 6 8
```

两个序列分别是 1 3 5 与 2 4 6 8，一共 7 个数。

合并结果按序为 1、2、3、4、5、6、8。

两个序列各自已经有序，所以每次只需要比较两边的「队首」，谁小谁先出队。

本题关键

① 两个序列都已有序，是本题最重要的条件。利用它可以把复杂度降到 O(n + m)。

② 做法：用两个指针分别指向两个序列的头部，每次取较小的那个放进结果，对应指针前进一格。

③ 一方先取完后，另一方可能还有剩余，必须用两个循环把剩下的补进结果——这是最容易漏的一步。

思路拆解：双指针归并

与其把两串数混在一起重新排序，不如利用「各自已有序」这个现成的条件。

观察 · 已完成的有序性

既然两个序列都排好了，那么合并后的最小值一定出现在两边的第一个元素之中。取出它之后，剩下的部分仍然满足同样的性质。

做法 · 两个指针各守一边

设两个指针分别指向两个序列的开头，比较两处的值，把较小者追加到结果序列，然后让对应的指针前进一格。重复这个过程。

收尾 · 谁没走完谁补上

当其中一个指针走到末尾时，另一个序列可能还剩若干元素。因为剩下的部分本身有序，直接依次追加到结果末尾即可。

对照 · 另一种写法

也可以把两个序列全部塞进一个 vector 再 sort 一次，结果同样正确，复杂度是 O((n+m) log(n+m))。本题数据不大时都行，但双指针更「懂事」。

一句话记住

题目白送的条件一定要用。「已经有序」这四个字，往往就是让你把 log 因子省掉的关键。双指针归并是后续学习归并排序、求逆序对、多路合并的基础，值得写熟。

参考代码：双指针归并


```
#include <iostream>
#include <vector>
using namespace std;

int main() {
  int n, m;
  cin >> n >> m;

  vector<int> a(n), b(m);
  for (int i = 0; i < n; i++) cin >> a[i];
  for (int i = 0; i < m; i++) cin >> b[i];

  vector<int> c;                // 合并结果
  int i = 0, j = 0;
  while (i < n && j < m) {
    if (a[i] <= b[j]) c.push_back(a[i++]);   // 哪边小先取哪边
    else              c.push_back(b[j++]);
  }
  while (i < n) c.push_back(a[i++]);        // 收尾：a 还有剩
  while (j < m) c.push_back(b[j++]);        // 收尾：b 还有剩

  for (int t = 0; t < (int)c.size(); t++) {
    if (t) cout << " ";
    cout << c[t];
  }
  cout << "
";
  return 0;
}
```

用小于等于保证稳定

比较时写 a[i] 小于等于 b[j] 就先取 a 的元素。这样当两边相等时优先取前者，合并结果更符合直觉，也和归并排序的稳定写法一致。

两个收尾循环不能省

主循环在任一方走完时就结束，剩下的元素还没处理。漏掉收尾循环，答案会少一截，这是本题最常见的错误。

push_back 用得恰到好处

结果序列的长度是 n 加 m，但我们不必提前算好。用 push_back 一路追加，长度自己就长起来了——这正是 vector 最舒服的地方。

提交提示

本题样例已通过验证。可以再试一方为空的情况（如 n 为 0 或 m 为 0），程序应当直接把另一方原样输出。

V05 | 按绝对值排序

第 4 课 · 排序算法深入

vector 与排序结合 ② · 难度 ★★

给 n 个整数，按绝对值从小到大排序输出。若两个数的绝对值相同，则保持它们在原序列中的先后顺序。

输入格式

第一行一个整数 n；第二行 n 个用空格隔开的整数，可能包含负数。

输出格式

一行，按绝对值升序输出这些数；绝对值相同时保持输入时的先后次序。

数据范围

$n \le 100000$；每个数的绝对值不超过 10^9。题目明说「保持先后」，这就是对稳定性的要求。

样例

```
5
-3 2 -1 4 -2
```

```
-1 2 -2 -3 4
```

各数的绝对值依次是 3、2、1、4、2。按绝对值升序应为 1、2、2、3、4。

绝对值相同的 2 与 -2：原序列中 2 在前，稳定排序后仍是 2 在前，得到 -1、2、-2、-3、4。

若用普通的 sort，这两个数的先后是不确定的，结果可能变成 -1、-2、2、-3、4。

本题关键

① 比较规则要自己写：比较两个数的绝对值大小，而不是它们本身。

② 题目要求「绝对值相同保持先后」，这就是稳定性要求，应当用 stable_sort 而不是 sort。

③ stable_sort 的写法与 sort 完全一样，只是函数名不同——本课基础专栏里讲过的那条规则，在这里正好用上。

思路拆解：自定义规则 + 稳定排序

这道题把「自定义比较器」与「稳定性」两个知识点合在了一起。

第一步 · 明确比较规则

题目要按绝对值排序，所以比较器里要比的是两个数的绝对值。绝对值小的排在前面。

第二步 · 识别稳定性要求

「绝对值相同保持先后」是原序列的顺序不能被打乱。普通 sort 不保证这一点，必须换成 stable_sort。

第三步 · 写法完全一样

stable_sort(v.begin(), v.end(), cmp) 与 sort 的调用形式一模一样，只是名字不同。记住这一条，本题就解决了一半。

第四步 · 验证与对照

拿样例验证：若把 stable_sort 换成 sort，样例很可能仍然输出正确结果——但隐藏数据里等值的先后一旦被打乱，答案就错了。这类错误不会在样例上暴露。

一句话记住

看到「相同的保持原顺序」，就该条件反射地想到 stable_sort。它与 sort 只有名字之差，却是「样例能过、提交挂掉」这类问题最常见的分水岭。

参考代码：stable_sort + 自定义比较器


```cpp
#include <iostream>
#include <vector>
#include <algorithm>
#include <cmath>
using namespace std;

// 按绝对值从小到大
bool cmp(int x, int y) {
  return abs(x) < abs(y);
}

int main() {
  int n;
  cin >> n;

  vector<int> v(n);
  for (int i = 0; i < n; i++) cin >> v[i];

  stable_sort(v.begin(), v.end(), cmp);   // 稳定排序：等值保序

  for (int i = 0; i < (int)v.size(); i++) {
    if (i) cout << " ";
    cout << v[i];
  }
  cout << "
";
  return 0;
}
```

cmp 只比绝对值

比较器里用 abs 取绝对值再比大小，与元素本身的正负无关。-3 与 3 在比较时地位相同，具体谁前谁后由稳定性决定。

stable_sort 的位置

它在 algorithm 头文件里，用法与 sort 完全一致：起止迭代器在前、比较器在后。把 sort 改成 stable_sort 即可，不必改任何其他代码。

abs 需要 cmath

abs 处理整数时来自 cstdlib，处理浮点时来自 cmath。为稳妥起见把 cmath 写上，编译器会自己挑合适的重载。

提交提示

本题样例已通过验证。请务必拿「两个数的绝对值相同」的数据自测，那才是这道题真正要考的地方。

专题：稳定性到底在说什么

很多排序题会明说「相同元素保持原来的先后」，这句话对应的就是稳定性。它不是抽象概念，而是直接决定答案对不对。

什么是稳定排序

给两个「比较结果相等」的元素，排序后如果它们的相对先后与原来一致，这个排序就是稳定的。

例如序列 [A5, B5]（两个都是 5，A 在前）。

稳定排序后仍是 A5 在前；不稳定排序可能变成 B5 在前。

在竞赛题里，稳定性的典型表现就是「值相同时按编号升序」——要么用稳定排序，

哪些稳定、哪些不稳定

稳定

插入排序 · 冒泡排序 · 归并排序 · stable_sort

不稳定

sort（快排实现） · 选择排序 · 堆排序

结论：默认的 sort 不要拿来处理「等值需保序」的需求。

题目要求保序时，三种写法任选一种

① 换用 stable_sort：写法与 sort 完全一样，只是名字不同。最省事，本课 V05 用的就是它。

② 在比较器里补一级关键字：值相同时比较原编号。本课 T04（生日）与 T05（分数线划定）都用了这一招。

③ 把下标当最后一级关键字整体排序：本课 T10（插入排序 P7910）采用的做法，也是「稳定」这个概念最直接的代码翻译。

错题门诊：本课六个高频坑

① 比较器方向写反

想降序却用了小于号，想升序却用了大于号。样例看起来「差不多对」，实际顺序全反。写完逐条对着题面核一遍方向。

② 把「长度」当作 sort 的参数

写成 sort(a, n) 编译直接报错。正确写法是给两个位置：sort(a, a + n)。这个概念错一次就记住了。

③ 比较器里用了小于等于

比较函数必须满足严格弱序，只能用小于或大于。写成小于等于会让 sort 的行为不可预测，轻则乱序、重则崩溃。

④ vector 下标越界

用 v[i] = x 给一个还没扩容的 vector 赋值，或者循环条件写成 i 小于等于 v.size()，都会越界。追加一律用 push_back。

⑤ 区间合并忘记收尾结算

扫描结束后最后一个覆盖段还没有参与比较。补上这一句，答案才完整——这是区间类题目最经典的漏项。

⑥ 该用 long long 的地方用了 int

求和、累加高度、大权值相加，这些地方 int 很容易溢出成负数。看到 10^9 级别的数据就要提高警惕。

课堂总结：把工具和套路带走

三块基础，三种武器

sort 负责「按规则重排」，结构体负责「把一条记录打包」，vector 负责「存不定长的数据」。三者组合起来，本课十五道题全部能落地。

十道例题，一条主线

所有排序题的骨架都一样：先想清楚「按什么排、优先级怎么分、方向是升还是降」，再把它写成比较器。题目千变万化，这四问不变。

两道难关，一个洞察

P7910 的关键是「改一个数，就只挪一个数」；V04/V05 的关键是「利用已有序」与「识别稳定性要求」。会问自己「有没有现成的有序性可用」，很多题就简单一半。

课后任务与下节预告

① 把十道例题的代码全部敲一遍，重点是三个比较器：多关键字、带输入序号、按绝对值。

② 五道 vector 练习至少手写 V04 与 V05，体会「利用有序」与「稳定排序」的差别。

③ 下节课预告：在有序序列上做查找——二分答案与贪心策略，排序打下的地基，从这里开始盖楼。

先排好，再让答案浮出来

把题面里的规则读清楚，写成一行比较器，排序题的骨架就立住了

课后任务：十道例题代码全部手敲一遍 · V04 与 V05 独立完成

第 5 课预告 · 二分答案与贪心策略

自主练习清单：十道题目与题号

第 4 课 · 排序算法深入

课后作业 · 建议按顺序完成

下面十道题按难度从易到难排列，题号即洛谷题号。建议先独立思考、再回看课堂对应页面，最后对照代码自测。

01	P1059	明明的随机数	★
	排序去重		
02	P2676	超级书架	★
	降序 + 贪心		
03	P1093	奖学金	★★
	多关键字排序		
04	P1104	生日	★★
	四级关键字 + 稳定性		
05	P1068	分数线划定	★★
	名次并列延伸		
06	P1781	宇宙总统	★★
	大数比较		
07	P5143	攀爬者	★★★
	结构体 + 几何距离		
08	P1204	挤牛奶	★★★
	区间合并		
09	P1583	魔法照片	★★★
	两轮排序 + 取模		
10	P7910	插入排序	★★★★
	动态维护名次		